Aprendizaje automático aplicado: teoría y práctica

Apunte del Taller #3 · Respuesta a una campaña bancaria · Diplomado en Data Science UC 2026

Este apunte tiene dos capas para cada tema. Primero, la **teoría general**: qué es, cómo funciona, sus fórmulas, ventajas y limitaciones, de modo que sirva para cualquier problema. Después, **qué pasa en el taller**: cómo se aplica a los datos del banco, el fragmento de código que lo hace y qué resultado obtuvimos.

#	Tema	Pregunta del taller
1	Tipos de aprendizaje y formulación del problema	1(a)
2	Fuga de información	1(a), 1(c), 2
3	Análisis exploratorio y calidad de datos	1(b)
4	Ingeniería de variables, codificación y escalamiento	1(c)
5	Validación cruzada y predicciones fuera de pliegue	2
6	Métricas para clasificación probabilística	2(b), 3
7	Selección de variables (filtro F de ANOVA)	2(a)
8	Regresión logística	2(a), 2(b)
9	Árboles de decisión y sobreajuste	2(b), 2(c)
10	Ensambles: bagging, Random Forest y boosting	2(b), 2(c)
11	Máquinas de vectores de soporte (SVM)	2(b), 2(c)
12	Naive Bayes	2(b), 2(c)
13	Discriminación vs calibración	2(c)
14	Teoría de decisión: umbrales y costos	3(a), 3(b)
15	Análisis de componentes principales (PCA)	4(a)
16	Agrupamiento K-means y silueta	4(b)
17	Detección de anomalías con Mahalanobis	5

1. Tipos de aprendizaje y formulación del problema

Teoría general

El aprendizaje automático se divide en dos grandes familias:

- **Supervisado:** cada observación tiene variables de entrada X y una respuesta conocida y. El objetivo es aprender una función $f(X) \approx y$ que generalice a casos nuevos. Si y es numérica es **regresión**; si es una categoría es **clasificación**.
- **No supervisado:** solo hay X. Se busca estructura: grupos (clustering), direcciones de variación (PCA) o casos atípicos (anomalías).

En clasificación binaria hay dos niveles de salida: la **probabilidad** estimada P(y = 1 | X) y la **decisión** (sí/no), que se obtiene comparando la probabilidad con un umbral. Separar ambos niveles es clave: el modelo estima, el negocio decide.

Formular bien un problema exige definir: la **unidad de observación** (qué es una fila), la **respuesta** y su clase positiva, el **momento de la predicción** (qué información existe en ese instante) y el **uso** que se dará a la predicción.

Qué pasa en el taller

- Tarea: clasificación binaria supervisada con salida probabilística. Unidad: un cliente con su llamada registrada (3617 en entrenamiento, 904 en prueba).
- Respuesta: *contrata* (1 = contrata el depósito, clase positiva). Momento: antes de la llamada. Uso: decidir a quién llamar.
- Las preguntas 4 y 5 cambian a aprendizaje no supervisado: PCA, K-means y anomalías, sin usar la respuesta.

2. Fuga de información (data leakage)

Teoría general

Hay fuga cuando el modelo usa, durante el entrenamiento o la evaluación, información que **no estaría disponible** en el momento real de predecir. El síntoma típico es un desempeño excelente en validación que se derrumba en producción. Hay dos formas:

- **Fuga por variables:** un predictor que es consecuencia de la respuesta o se mide después de ella.
- **Fuga por procedimiento:** aprender transformaciones (medias, desviaciones, categorías, selección de variables) usando también las filas con que se evaluará. La regla es que *todo lo que se ajusta* debe ajustarse solo con datos de entrenamiento, dentro de cada pliegue.

Qué pasa en el taller

- **duracion** es fuga por variable: se conoce al terminar la llamada. Es la más asociada a contratar (554 s vs 228 s en promedio), y por eso mismo se excluye.
- Para evitar fuga por procedimiento, el preparador (estandarización + one-hot) y la selección F se ajustan dentro de cada pliegue, y el conjunto de prueba no se abre hasta 3(b), después de haber fijado modelo y umbral.

```
EXCLUIDAS = ['duracion', 'id_cliente', 'pliegue', 'contrata']
predictores_originales = [c for c in entrenamiento.columns if c not in EXCLUIDAS]
```

3. Análisis exploratorio y calidad de datos

Teoría general

Antes de modelar se revisa: tipos de variables, valores faltantes (explícitos como NA o implícitos como "unknown", 999, −1), duplicados, rangos plausibles, distribuciones (asimetría, colas, ceros) y el **balance de clases**.

Con clases desbalanceadas, la **exactitud** es engañosa: si el 90 % es negativo, un clasificador que siempre dice "no" logra 90 % de exactitud sin aportar nada. Por eso se prefieren métricas sensibles a la clase minoritaria (sensibilidad, AUC) y a la calidad de las probabilidades (log loss, Brier).

Los **códigos especiales** deben interpretarse según el dominio: un −1 que significa "no aplica" no es un número, y tratarlo como tal introduce relaciones falsas (el modelo creería que −1 días está "cerca" de 0 días).

Qué pasa en el taller

Revisión	Hallazgo	Decisión
Faltantes NA	0	Nada que imputar.
"unknown"	resultado_previo 82,4 %, canal 30,1 %, educación 4,3 %, ocupación 1,0 %	Mantener como categoría (puede ser informativo).
Duplicados	0 filas, 0 id repetidos	Se conservan todas las filas.
Balance	88,5 % no / 11,5 % sí; igual en los 5 pliegues	No usar exactitud como criterio.
saldo < 0	302 registros (mín. −3313 €)	Sobregiros: dato válido.
dias_previos = −1	2979; coincide con 0 contactos previos	Código: crear indicador (tema 4).

Resultado en el taller
Exactitud de predecir siempre "no contrata": 88,5 %, con sensibilidad 0. Justifica usar log loss, Brier, AUC y costos.

4. Ingeniería de variables, codificación y escalamiento

Teoría general

Ingeniería de variables es transformar o crear predictores para que representen mejor el fenómeno. Técnicas habituales:

- **Indicadores** para separar un estado cualitativo de una cantidad (p. ej., "tuvo contacto previo" vs "días desde el contacto").
- **Transformación logarítmica** log(1 + x) para variables de conteo o monto con cola larga: reduce la asimetría, el peso de extremos y modela rendimientos decrecientes. El +1 permite x = 0.

Codificación one-hot. Una variable categórica con K categorías se transforma en columnas binarias. Si se crean K columnas y el modelo tiene intercepto, la suma de las K columnas es siempre 1: **colinealidad perfecta** (la "trampa de las dummies"). Se resuelve dejando $K - 1$ columnas (una categoría de referencia) o usando regularización.

Estandarización. Lleva cada variable a media 0 y desviación 1:

$$z = (x - \bar{x}) / s$$

Familia de métodos	¿Sensible a la escala?	Razón
Basados en distancias: SVM con kernel, K-means, PCA, Mahalanobis, k-NN	Sí	La variable de mayor rango domina la distancia o la varianza.
Modelos lineales con penalización (Lasso, Ridge)	Sí	La penalización compara magnitudes de coeficientes.
Regresión logística sin penalización	No en las predicciones	Escalar solo reescala coeficientes; ayuda a la convergencia numérica.
Árboles y ensambles de árboles	No	Las divisiones "x ≤ c" no cambian con transformaciones monótonas.
Naive Bayes gaussiano	No	Estima media y varianza por columna; se ajusta a cualquier escala.

Qué pasa en el taller

```
def construir_variables(df):
    d = df.copy()
    d['contactado_antes'] = (d['dias_previos'] != -1).astype(int)
    d['log_contactos_previos'] = np.log1p(d['contactos_previos'])
    d['dias_previos'] = d['dias_previos'].replace(-1, 0)
    return d

def nuevo_preparador():
    return ColumnTransformer([
        ('num', StandardScaler(), NUM),          # 6 numéricas estandarizadas
        ('bin', 'passthrough', BIN),             # contactado_antes (0/1)
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore',
                              sparse_output=False), CAT)])   # 9 categóricas
```

- contactado_antes captura un salto real: contrata el 22,4 % de los contactados antes vs 9,2 % del resto.
- log_contactos_previos reduce la asimetría de 5,85 a 2,51; reemplaza a la variable original para no duplicar información.
- drop='first' evita la trampa de las dummies en la logística sin penalización; "unknown" queda como categoría propia.

- Una sola matriz sirve a todos los modelos: numéricas estandarizadas (necesario para SVM) y dummies 0/1 (los árboles y NB no se ven afectados).

Resultado en el taller
p = 42 columnas preparadas: 6 numéricas, 1 indicador y 35 dummies. El preparador se ajusta de nuevo en cada pliegue.

5. Validación cruzada y predicciones fuera de pliegue (OOF)

Teoría general

El error de entrenamiento es optimista: el modelo ya vio esos datos. Para estimar el error en datos nuevos se usa **validación cruzada k-fold**: se divide la muestra en k pliegues; en cada ronda se entrena con $k - 1$ y se predice el pliegue restante. Al final cada observación tiene una predicción **fuera de pliegue (OOF)**, hecha por un modelo que no la vio. Las métricas se calculan sobre el vector completo de predicciones OOF.

- **Estratificación:** mantener la proporción de clases en cada pliegue (importante con desbalance).
- **Pipeline dentro del pliegue:** todo paso que aprende de los datos se ajusta dentro de cada ronda.
- **Tres conjuntos con roles distintos:** entrenamiento (ajustar), validación (elegir modelo, hiperparámetros, umbral) y prueba (estimar una sola vez el desempeño final).

Qué pasa en el taller

```
def validar(crear_modelo, k_sel=None, puntaje=False):
    oof = np.zeros(len(datos))
    for f in sorted(np.unique(pliegues)):          # 5 pliegues entregados
        aj, va = pliegues != f, pliegues == f
        prep = nuevo_preparador().fit(datos[aj])    # aprende SOLO con ajuste
        Xa, Xv = prep.transform(datos[aj]), prep.transform(datos[va])
        if k_sel:                                    # selección también dentro
            sel = SelectKBest(f_classif, k=k_sel).fit(Xa, y[aj])
            Xa, Xv = sel.transform(Xa), sel.transform(Xv)
        m = crear_modelo(Xa.shape[1]).fit(Xa, y[aj])
        oof[va] = m.decision_function(Xv) if puntaje else m.predict_proba(Xv)[:, 1]
    return oof
```

Los pliegues vienen definidos en la columna *pliegue* y están estratificados (11,5–11,6 % de positivos en cada uno). Todos los modelos pasan por la misma función, lo que garantiza una comparación justa.

6. Métricas para clasificación probabilística

Teoría general

Log loss (entropía cruzada). Penaliza el logaritmo de la probabilidad asignada a lo que realmente ocurrió. Castiga fuertemente la sobreconfianza equivocada (si $p \to 0$ y ocurre $y = 1$, la pérdida tiende a infinito). Es una **regla de puntuación propia**: se minimiza reportando las probabilidades verdaderas.

$$\text{LogLoss} = -(1/n)\ \Sigma_i\ [\ y_i \log p_i + (1 - y_i) \log(1 - p_i)\ ]$$

Brier score. Error cuadrático medio de la probabilidad. También es regla propia, pero más tolerante con errores extremos.

$$\text{Brier} = (1/n)\ \Sigma_i\ (p_i - y_i)^2$$

AUC (área bajo la curva ROC). La curva ROC grafica sensibilidad vs 1 − especificidad al mover el umbral. El AUC equivale a la probabilidad de que un positivo elegido al azar reciba un puntaje mayor que un negativo. Solo depende del **orden**: no cambia si se aplica cualquier transformación monótona a los puntajes. 0,5 = azar, 1 = perfecto.

$$\text{AUC} = P(\ s(X^+) > s(X^-)\ )$$

Matriz de confusión (para un umbral dado) y métricas derivadas:

	Predicho 0	Predicho 1
Real 0	TN (verdadero negativo)	FP (falso positivo)
Real 1	FN (falso negativo)	TP (verdadero positivo)

$$\text{Sensibilidad} = TP / (TP + FN)$$

$$\text{Especificidad} = TN / (TN + FP)$$

Modelo de referencia. Predecir a todos la tasa base es el punto de partida: un modelo útil debe tener menor log loss y Brier que él. Su AUC es 0,5 porque no ordena.

Qué pasa en el taller

```
def metricas(y_true, p):
    p = np.clip(p, 1e-15, 1 - 1e-15)        # evita log(0) con hojas puras
    return {'log_loss': log_loss(y_true, p), 'brier': brier_score_loss(y_true, p),
            'auc': roc_auc_score(y_true, p)}
```

- Referencia OOF: log loss 0,357, Brier 0,102, AUC 0,50. El mejor modelo baja el log loss a 0,311 y sube el AUC a 0,744.
- Para la SVM solo se informa AUC, porque su salida es un puntaje, no una probabilidad.
- Se elige el modelo por log loss porque la pregunta 3 decide con probabilidades y costos: interesa que las probabilidades sean creíbles.

7. Selección de variables

Teoría general

Seleccionar variables busca modelos más simples, interpretables y, a veces, con menos sobreajuste. Hay tres familias:

- **Filtro:** ordena variables con un estadístico univariado (F de ANOVA, chi-cuadrado, información mutua) sin entrenar el modelo final. Rápido, pero ciego a redundancia e interacciones.

- **Envoltura (wrapper):** evalúa subconjuntos entrenando el modelo (selección hacia adelante/atrás, RFE). Más preciso y costoso.
- **Embebida:** la selección ocurre al entrenar (Lasso, importancia en árboles).

El **estadístico F de ANOVA**, para una columna y una respuesta binaria, compara la variabilidad *entre* las medias de las clases con la variabilidad *dentro* de las clases:

$$F = [\ \Sigma_g\ n_g\ (\bar{x}_g - \bar{x})^2 / (G - 1)\ ] / [\ \Sigma_g\ \Sigma_{i \in g}\ (x_i - \bar{x}_g)^2 / (n - G)\ ]$$

Selección vs PCA: la selección conserva columnas originales y es supervisada; PCA crea combinaciones lineales nuevas de todas las columnas y es no supervisado (maximiza varianza, no capacidad predictiva).

Qué pasa en el taller

```
oof_10, _ = validar(logit, k_sel=10)   # SelectKBest(f_classif, k=10) dentro de cada pliegue
```

- Columnas elegidas en los 5 pliegues: dias_previos, log_contactos_previos, contactado_antes, resultado_previo_success, resultado_previo_unknown, canal_unknown, hipoteca_yes, mes_may, mes_oct; mes_mar en 4 de 5 y estado_civil_married en 1.
- La selección es estable pero redundante: cuatro columnas codifican "hubo campaña previa".

Regresión logística	Log loss	Brier	AUC
42 columnas	0,3151	0,0895	0,7250
10 columnas F	0,3203	0,0914	0,7011

Resultado en el taller
Seleccionar empeora las tres métricas: el filtro deja fuera información útil (otros meses, préstamo, contactos de campaña) e incluye columnas redundantes.

8. Regresión logística

Teoría general

Modela el logaritmo de las chances (log-odds) como una combinación lineal de los predictores y lo transforma en probabilidad con la función sigmoide:

$$\log[\, p / (1 - p) \,] = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$$

$$p = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p)})$$

- Los coeficientes se estiman por **máxima verosimilitud**, que equivale a minimizar el log loss en entrenamiento. Por eso suele producir probabilidades bien calibradas.
- Interpretación: e^{β_j} es el factor por el que se multiplican las chances al aumentar x_j en una unidad, con lo demás fijo.
- Limitaciones: frontera lineal (no capta interacciones ni no linealidades si no se crean), sensible a colinealidad, y sin penalización puede inestabilizarse con muchas columnas o categorías raras (separación). La penalización (Ridge/Lasso) controla esto.

Qué pasa en el taller

```
def logistica_sin_penalizacion():
    if sklearn >= 1.8:  return LogisticRegression(C=np.inf, max_iter=10000)
    else:               return LogisticRegression(penalty=None, max_iter=10000)
```

Resultado en el taller
Logística con 42 columnas: log loss 0,315, Brier 0,0895 (empata con el mejor), AUC 0,725. Es tercera: bien calibrada, pero ordena peor que los ensambles porque no capta interacciones.

9. Árboles de decisión y sobreajuste

Teoría general

Un árbol divide recursivamente el espacio con preguntas del tipo "$x_j \le c$". En cada nodo elige la división que más reduce la **impureza** (Gini o entropía). La probabilidad predicha en una hoja es la proporción de positivos en ella.

$$\text{Gini} = 1 - \Sigma_k \, p_k^{\,2}$$

$$\text{Entropía} = -\Sigma_k \, p_k \log p_k$$

- **Ventajas:** interpretable, capta no linealidades e interacciones, no requiere escalar.
- **Desventajas:** alta varianza (pequeños cambios en los datos cambian el árbol) y probabilidades "en escalones" (pocas hojas = pocos valores).
- **Compromiso sesgo–varianza:** más profundidad reduce el sesgo pero aumenta la varianza. Se controla con profundidad máxima, mínimo por hoja o poda.
- **Diagnóstico de sobreajuste:** brecha creciente entre el error de entrenamiento y el de validación.

Qué pasa en el taller

```
DecisionTreeClassifier(max_depth=3, min_samples_leaf=20)   # y max_depth=6
```

Árbol	Log loss entren.	Log loss OOF	Brecha	AUC OOF
Profundidad 3	0,320	0,330	0,010	0,636
Profundidad 6	0,302	0,339	0,037	0,664

Resultado en el taller
El árbol 6 ajusta mejor el entrenamiento pero peor la validación en log loss: sobreajuste. A la vez ordena algo mejor (AUC): el árbol 3 tiene muy pocas hojas y poca resolución. Ningún árbol solo compite con los ensambles.

10. Ensambles: bagging, Random Forest y boosting

Teoría general

Un ensamble combina muchos modelos débiles o inestables para obtener uno más preciso.

Bagging (bootstrap aggregating): entrena B árboles profundos sobre muestras bootstrap y promedia sus predicciones. Reduce la **varianza**. Si los árboles tienen varianza σ^2 y correlación ρ entre sí, la varianza del promedio es:

$$\text{Var(promedio)} = \rho\sigma^2 + (1 - \rho)\sigma^2 / B$$

Con B grande el segundo término desaparece, pero $\boldsymbol{\rho\sigma^2}$ **queda**: si los árboles se parecen (todos usan las mismas variables fuertes), el promedio mejora poco.

Random Forest agrega aleatoriedad en cada división: solo se consideran m columnas al azar (típicamente $m = \sqrt{p}$). Eso **reduce ρ** (decorrelaciona los árboles) y mejora el promedio. Además entrega importancia de variables y error OOB.

Boosting es secuencial: cada árbol pequeño se ajusta a los errores (gradiente de la pérdida) del ensamble actual y se suma con una tasa de aprendizaje ν:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

- Reduce principalmente el **sesgo**. Hiperparámetros clave: número de árboles M, profundidad, tasa ν (pequeña = aprendizaje lento y más robusto).
- Puede sobreajustar si M es muy grande o ν muy alto; se controla con validación, submuestreo o detención temprana.
- Variantes: AdaBoost (repondera observaciones), Gradient Boosting (ajusta gradientes), XGBoost/LightGBM (implementaciones optimizadas).

	Bagging	Random Forest	Boosting
Construcción	Paralela	Paralela	Secuencial
Árboles	Profundos	Profundos, con m columnas por división	Pequeños
Reduce	Varianza	Varianza (más que bagging)	Sesgo (y algo de varianza)
Riesgo	Árboles correlacionados	Bajo	Sobreajuste si no se regula

Qué pasa en el taller

```
RandomForestClassifier(n_estimators=200, max_features='sqrt', min_samples_leaf=10, random_state
=2026)
GradientBoostingClassifier(n_estimators=200, max_depth=3, learning_rate=0.05, random_state=2026
)
```

Resultado en el taller

Gradient Boosting (log loss 0,311; AUC 0,744) y Random Forest (0,312; 0,740) son los dos mejores modelos. Ambos captan interacciones como mes × canal o resultado previo × contacto, que la logística no ve. Se elige Gradient Boosting por menor log loss OOF.

11. Máquinas de vectores de soporte (SVM)

Teoría general

La SVM busca la frontera que separa las clases con el **máximo margen**. Con datos no separables, permite violaciones con un costo C. El **truco del kernel** permite fronteras no lineales calculando similitudes en un espacio transformado sin construirlo. El kernel radial (RBF) es:

$$K(x, x') = \exp(\, -\gamma \, \|x - x'\|^2 \,)$$

- **C**: costo de los errores. C alto = frontera ajustada, menos regularización (más varianza); C bajo = margen amplio (más sesgo).
- **γ**: alcance de cada punto. γ alto = influencia local, fronteras irregulares (sobreajuste); γ bajo = frontera casi lineal.
- **Escalamiento:** el kernel usa distancias euclidianas, por lo que es indispensable estandarizar.
- **Salida:** un puntaje de decisión (distancia con signo a la frontera), no una probabilidad. Para obtener probabilidades hay que calibrar (Platt o isotónica).

Qué pasa en el taller

```
SVC(kernel='rbf', C=1.0, gamma=1/p)   # p = 42; se usa decision_function()
```

AUC OOF	γ = 0,1/p	γ = 1/p	γ = 10/p
C = 0,1	0,628	0,690	0,700
C = 1	0,652	0,685	0,700
C = 10	0,676	0,687	0,691

Resultado en el taller
Con la configuración pedida, AUC 0,685. γ muy bajo empeora (frontera demasiado simple); C influye poco. Sin estandarizar las numéricas el AUC cae a 0,599: el saldo domina las distancias.

12. Naive Bayes

Teoría general

Aplica el teorema de Bayes con el supuesto "ingenuo" de **independencia condicional** de las variables dada la clase:

$$P(y \mid x) \propto P(y) \cdot \Pi_j P(x_j \mid y)$$

- La variante **gaussiana** supone que cada x_j, dentro de cada clase, sigue una normal con su propia media y varianza.
- **Ventajas:** muy rápido, funciona con pocos datos, suele ordenar razonablemente bien.
- **Problema:** si las variables están correlacionadas, el producto "cuenta la misma evidencia varias veces" y las probabilidades se van a los extremos (0 o 1): mala calibración.
- **Variables binarias:** una columna 0/1 no es normal. Con dummies raras la varianza estimada es casi cero y la densidad gaussiana explota. Para ellas corresponde una variante Bernoulli o categórica.

Qué pasa en el taller

```
GaussianNB()   # sobre las 42 columnas: numéricas estandarizadas + binarias
```

Resultado en el taller

Log loss 2,256 y Brier 0,165: **peor que el modelo de referencia**, aunque su AUC (0,702) es aceptable.

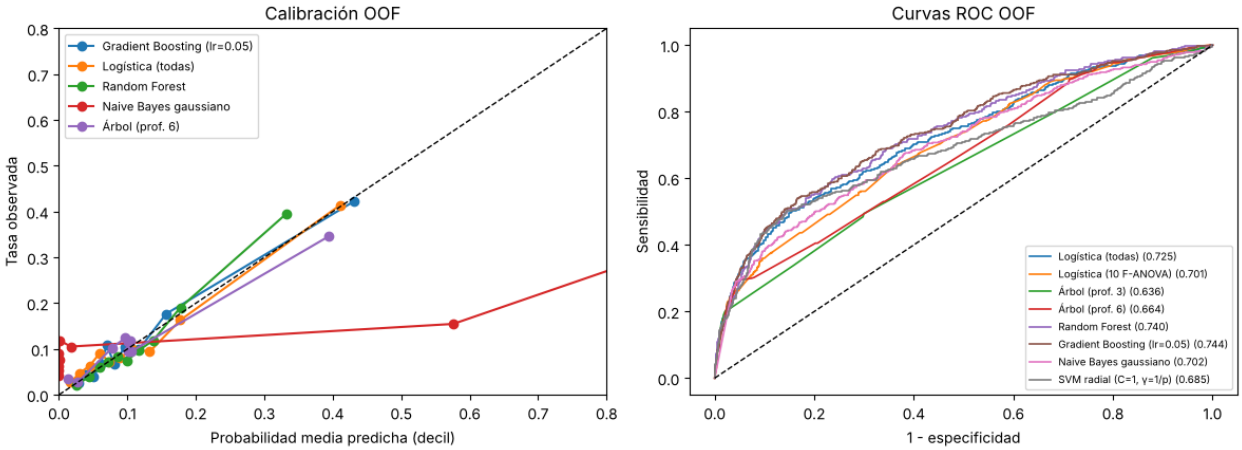
El 75 % de sus probabilidades OOF son menores que 0,01 y el 11 % mayores que 0,99; su promedio es 0,16 (tasa real 0,115). Causa: columnas redundantes (dias_previos, contactado_antes, log_contactos_previos, resultado_previo_unknown) y dummies tratadas como normales.

13. Discriminación vs calibración

Teoría general

- **Discriminación** = capacidad de ordenar (¿los positivos tienen puntajes más altos?). Se mide con AUC.
- **Calibración** = que las probabilidades signifiquen lo que dicen: entre los casos con p ≈ 0,2, ocurre el evento en ~20 %. Se evalúa con curvas de calibración y la contribuyen log loss y Brier.
- Un modelo puede ordenar perfecto y estar mal calibrado (por ejemplo, multiplicar todas las probabilidades por 0,5 no cambia el AUC pero arruina el log loss).
- Para decidir con umbrales y costos se necesitan probabilidades calibradas. Si un modelo discrimina bien pero calibra mal, se puede recalibrar (Platt, isotónica).

Qué pasa en el taller



Calibración OOF por deciles (izquierda) y curvas ROC (derecha). Boosting, Random Forest y la logística siguen la diagonal; Naive Bayes se aleja.

Resultado en el taller

Naive Bayes y la logística con 10 columnas tienen AUC parecido (0,70), pero el log loss de Naive Bayes es siete veces mayor. Mejor ordenamiento no implica mejores probabilidades.

14. Teoría de decisión: umbrales y costos

Teoría general

Convertir una probabilidad en decisión requiere un umbral t: se predice positivo si p ≥ t. Si un falso positivo cuesta C_{FP} y un falso negativo C_{FN}, conviene actuar cuando el costo esperado de no actuar supera al de actuar:

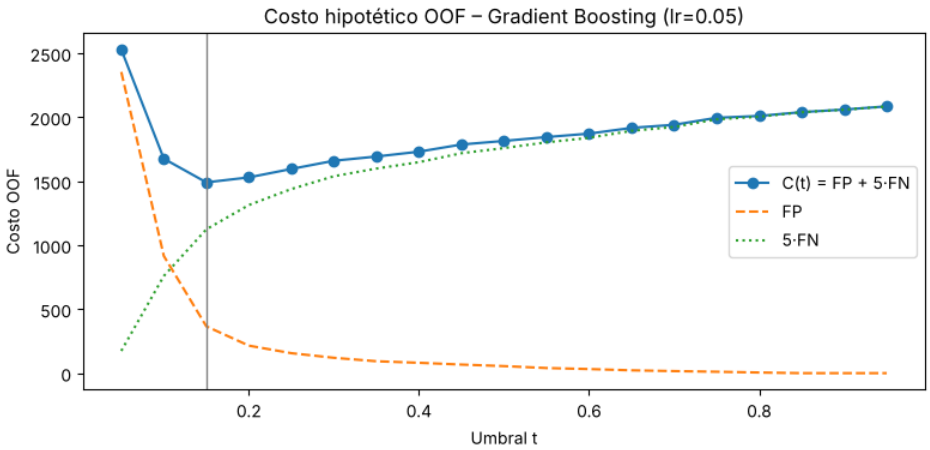
$$p \cdot C_{FN} > (1 - p) \cdot C_{FP} \Rightarrow t^* = C_{FP} / (C_{FP} + C_{FN})$$

- Con costos iguales, t* = 0,5. Si un FN cuesta más, el umbral baja: se acepta más FP para perder menos positivos.
- Este umbral teórico vale si las probabilidades están calibradas. En la práctica se elige empíricamente minimizando el costo en validación (OOF), nunca en prueba.
- Bajar el umbral sube la sensibilidad y baja la especificidad; la curva ROC muestra todo ese recorrido.

Qué pasa en el taller

```
for t in np.arange(0.05, 0.951, 0.05):
    pred = (p_oof >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y, pred).ravel()
    costo = fp + 5 * fn
```

Con C_{FP} = 1 y C_{FN} = 5, el umbral teórico es 1/6 ≈ 0,167. El mínimo empírico OOF está en **t = 0,15** (costo 1492 vs 1816 con t = 0,5), consistente con un modelo bien calibrado.



Costo OOF según umbral: a la izquierda dominan los FP, a la derecha los FN (multiplicados por 5).

Evaluación final en prueba (3b). Se reentrena con todo el entrenamiento y se evalúa una vez:

Regla	TP	FP	FN	Sensib.	Especif.	C/n
Boosting, t = 0,15	55	112	49	52,9 %	86,0 %	0,395
Boosting, t = 0,50	21	16	83	20,2 %	98,0 %	0,477
Referencia	0	0	104	0 %	100 %	0,575

Resultado en el taller

Métricas en prueba: log loss 0,309, Brier 0,089, AUC 0,750, muy similares a las OOF: la validación fue un buen estimador.

Con t = 0,15 se llama al 18 % de los clientes y se detecta al 53 % de los que contratan; 1 de cada 3 llamadas termina en contratación (vs 11,5 % al azar).

Pendiente de verificar: costos reales, estabilidad temporal y efecto causal de la llamada.

15. Análisis de componentes principales (PCA)

Teoría general

PCA busca direcciones ortogonales (componentes) que capturan sucesivamente la máxima varianza. Con variables estandarizadas, las componentes son los **vectores propios** de la matriz de correlación R, y sus varianzas son los **valores propios** λ:

$$R\, v_k = \lambda_k\, v_k$$

$$\text{Proporción explicada por } PC_k = \lambda_k \,/\, \Sigma_j\, \lambda_j$$

- Las **cargas** (elementos de v_k) indican cuánto pesa cada variable en la componente. El signo es arbitrario.
- Estandarizar es obligatorio si las variables están en escalas distintas; si no, la de mayor varianza domina.
- PCA comprime bien solo si las variables están correlacionadas. Con variables independientes, cada componente explica ≈ 1/p.
- Es lineal, sensible a valores extremos y no supervisado: las componentes de mayor varianza no son necesariamente las más predictivas.

Qué pasa en el taller

```
VARS4 = ['edad', 'saldo', 'contactos_campana', 'contactos_previos']
Z = (entrenamiento[VARS4] - entrenamiento[VARS4].mean()) / entrenamiento[VARS4].std(ddof=1)
pca = PCA().fit(Z);  scores = pca.transform(Z)
```

	PC1	PC2
% varianza	27,7 %	26,7 % (acum. 54,4 %)
edad	0,58	−0,43
saldo	0,63	−0,31
contactos_campana	−0,38	−0,58
contactos_previos	0,35	0,62

Resultado en el taller
Las cuatro variables casi no se correlacionan (máx. 0,10), así que cada componente explica ≈ 25 % y el plano conserva solo el 54 %. PC1 contrapone edad y saldo a la insistencia en la campaña; PC2, el historial previo a los contactos actuales. Las clases contrata 0/1 se superponen por completo en el plano.

16. Agrupamiento K-means y silueta

Teoría general

K-means divide n observaciones en k grupos minimizando la suma de distancias cuadradas de cada punto a su centroide (WSS):

$$WSS = \Sigma_k\, \Sigma_{i \in C_k}\, \|x_i - \mu_k\|^2$$

- Algoritmo de Lloyd: asignar cada punto al centroide más cercano, recalcular centroides, repetir hasta converger.
- Converge a un óptimo **local** que depende del inicio: por eso se usan varias inicializaciones (n_init) y se conserva la de menor WSS.
- Supone grupos esféricos de tamaño similar, es sensible a la escala (estandarizar) y a valores extremos.

- WSS siempre baja al aumentar k, así que no sirve sola para elegir k. Se usa el codo o la **silueta**:

$$s(i) = (b(i) - a(i)) / \max\{ a(i), b(i) \}$$

donde a(i) es la distancia media de i a su propio grupo y b(i) la distancia media al grupo vecino más cercano. s ≈ 1: bien asignado; s ≈ 0: en la frontera; s < 0: mal asignado.

Qué pasa en el taller

```
KMeans(n_clusters=k, n_init=30, random_state=2026).fit(Z)   # k = 2, 3, 4 ; silhouette_score(Z, labels)
```

k	WSS	Silueta	Tamaños
2	11 893	0,301	2213 / 1404
3	9 685	0,334	2090 / 1348 / 179
4	7 665	0,363	2004 / 1289 / 177 / 147

Grupo (k = 4)	n	Perfil	% contrata
Jóvenes, saldo bajo	2004	Edad 34, sin campañas previas	10,8 %
Mayores con más saldo	1289	Edad 52, saldo medio 2146 €	11,4 %
Con historial previo	177	6,6 contactos previos	26,6 %
Muy insistidos	147	14,5 contactos en la campaña	4,8 %

Ojo, error típico
Los grupos no son clases predichas: K-means no usó la respuesta, y en ningún grupo contrata la mayoría. Sirven para describir y segmentar, no para clasificar.

17. Detección de anomalías con distancia de Mahalanobis

Teoría general

La distancia de Mahalanobis mide qué tan lejos está una observación del centro de la nube, ajustando por la escala y la correlación de las variables:

$$D^2(x) = (x - \mu)^T S^{-1} (x - \mu)$$

- Si las variables están incorrelacionadas y estandarizadas, D^2 es la suma de los z^2 de cada variable.
- Si los datos son **normales multivariados** con p variables, $D^2 \sim \chi^2$ con p grados de libertad. Usar el cuantil 0,99 como corte marcaría ≈ 1 % de los datos.
- Con distribuciones asimétricas o de colas pesadas, la proporción marcada supera ese 1 %. Además, la media y covarianza se estiman con los mismos atípicos (efecto enmascaramiento); alternativas robustas: MCD (determinante de covarianza mínima) o transformar las variables.
- "Atípico" es una afirmación estadística, no una prueba de error. Antes de eliminar, verificar con la fuente y evaluar el impacto.

Qué pasa en el taller

```
S = np.cov(Z, rowvar=False, ddof=1);  S_inv = np.linalg.inv(S)
d2 = np.einsum('ij,jk,ik->i', Zc, S_inv, Zc)
corte = stats.chi2.ppf(0.99, df=4)        # 13,28
```

id	Edad	Saldo	Contactos campaña	Contactos previos	D^2	Por qué
1477	60	71 188 €	1	0	518,6	Saldo a 22,8 desviaciones
1542	35	280 €	50	0	224,3	50 llamadas: 14,9 desv.

Resultado en el taller

Se marcan 189 registros (5,2 %), cinco veces el 1 % teórico, porque las variables tienen colas muy pesadas (curtosis del saldo ≈ 99, de contactos ≈ 40–51) y son conteos discretos.

Ambos casos son extremos en una sola variable y son plausibles (un jubilado con alto saldo, rellamadas automáticas). Se conservan; antes de eliminar se verificaría con el sistema contable y el call center.

Síntesis: el hilo conductor del taller

- **Definir bien el problema** (qué se predice, cuándo y para qué) determina qué variables son válidas: por eso duración queda fuera.
- **Explorar** revela códigos, desbalance y asimetrías que guían la preparación.
- **Preparar dentro de cada pliegue** y comparar con OOF da estimaciones honestas; la prueba confirma (0,744 vs 0,750 de AUC).
- **Elegir por log loss** porque se decidirá con probabilidades; distinguir ordenar (AUC) de calibrar.
- **El umbral lo fijan los costos**, no la costumbre: 0,15 en vez de 0,50 reduce el costo un 17 %.
- **Lo no supervisado describe** (perfiles, atípicos) y complementa la decisión, pero no reemplaza al modelo predictivo.